

110.10. La fonction `spriteDerriere()` — un lutin derrière le décor

`#include <spriteDerriere>` : `spriteDerriere(numero, x, y, tuile)`, c'est `sprite()`, mais derrière le décor.

1. Ce qu'il faut comprendre

Ce qu'elle fait : elle place un lutin comme `sprite()`, mais **derrière le décor**, comme `sprite(numero, x, y, tuile, DERRIERE)`. Plus besoin de retenir le 5e argument.

La ligne à écrire : `#include <spriteDerriere>`. Pas besoin d'inclure aussi `sprite`.

Ses arguments : le numéro du lutin, `x` et `y` en pixels, la tuile. Le décor ne cache le lutin que là où il est dessiné : sur le vide, on voit le lutin.

Ce qu'elle coûte : rien de plus qu'un `sprite()` : le compilateur la remplace par `sprite(..., DERRIERE)`.

Essaie : mets le héros à `x = 40`, sur le vide : il réapparaît.

On retrouve `spriteDerriere` dans 2 leçons

[110.11](#), [110.14](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **262 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	40 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	32 o	← CopierTuiles — 0 lettre sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	× 1	24 o sur place, dans <code>main()</code>
<code>poser()</code>	× 1	6 o sur place, dans <code>main()</code>
<code>image()</code>	× 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <spriteDerriere> : un lutin derrière le décor ----
// Un mur, et un héros posé dessus avec spriteDerriere() : le mur le cache.

#include <Tuile>
#include <poser>
#include <spriteDerriere> // spriteDerriere() : sprite(), mais derrière le décor
```

2. UN DESSIN — MUR

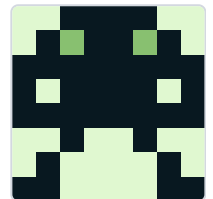
```
Tuile MUR = { // des briques (-), des joints (#) : aucun point
  "#####", "---#---", "---#---", "#####",
  "-#-----", "-#-----", "#####", "---#---",
};
```



à l'écran

3. UN DESSIN — HEROS

```
Tuile HEROS = { // les points (.) sont transparents
  "..####.", ".#-##-#.", "#####", "#.####.#",
  "#####", "..#..#..", ".#....#.", "##....##",
};
```



à l'écran

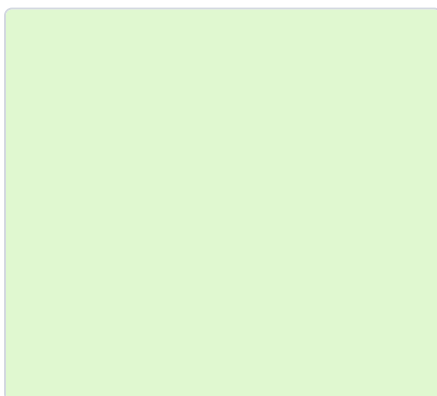
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  poser(9, 8, MUR); // le mur, case (9, 8)
  spriteDerriere(0, 72, 64, HEROS); // le héros, pile sur le mur : derrière lui

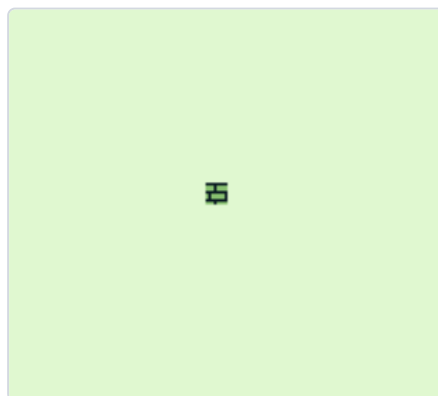
  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un mur de briques, seul : le héros est derrière.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin porte DERRIERE

✓ on voit la brique, pas le héros (nuance 1)

Ensuite : **110.11. Devant, derrière : le buisson sur Game Boy normale, devant puis derrière** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++